

INFORME TÉCNICO

CapyGuardian v2.1 — Nodo ROS 2 de navegación autónoma

Análisis de arquitectura: percepción LiDAR, detección de formas (paredes/cajas/esquinas), control PID y máquina de estados

Campo	Detalle
Archivo analizado	capytown_guardian.py (versión 2.1)
Framework	ROS 2 Humble
Sensor	LiDAR MS200 (360°)
Plataforma	Yahboom MicroROS-Pi5
Líneas de código	1225
Lenguaje	Python 3 (rclpy)

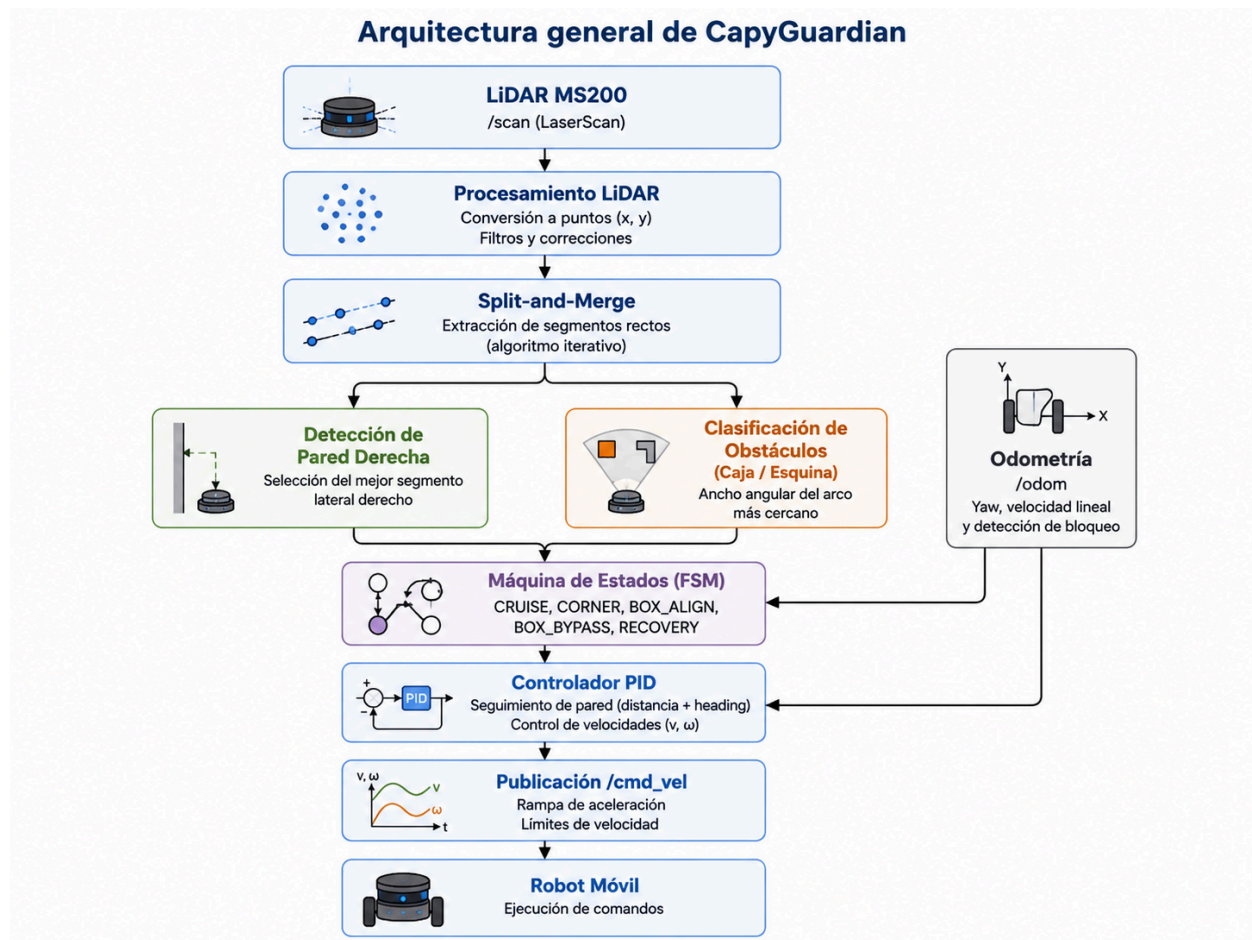
1. Resumen ejecutivo

CapyGuardian es un nodo único de ROS 2 que resuelve el reto de recorrer un anillo (circuito cerrado tipo pista) delimitado por paredes, esquivando cajas colocadas como obstáculos. El nodo integra en un mismo componente tres capas clásicas de robótica móvil:

- Percepción: procesa la nube de puntos del LiDAR y extrae segmentos de recta mediante el algoritmo Split-and-Merge, de donde se derivan la pared lateral derecha y la clasificación del obstáculo frontal (nada / caja / esquina).
- Control: un lazo PID completo (posición + heading + derivativo filtrado + integral con anti-windup) que sigue la pared derecha manteniendo una distancia objetivo.
- Decisión: una máquina de estados finitos (FSM) con persistencia de transiciones, timeouts y recuperación automática ante bloqueos.

El diseño más relevante del archivo es cómo distingue una CAJA de una ESQUINA usando exclusivamente el ancho angular del arco de puntos más cercano detectado al frente del robot, sin necesidad de un clasificador de imágenes ni de aprendizaje automático.

1.1. Arquitectura general de CapyGuardian



2. Arquitectura general del nodo

Todo el comportamiento vive en la clase CapyGuardian(Node). El ciclo de vida es:

- `__init__`: declara ~60 parámetros ROS configurables, crea las suscripciones (`/scan`, `/odom`), el publicador (`/cmd_vel`) y dos timers (loop de control y watchdog de arranque).
- `cb_scan(msg)`: callback del LiDAR. Convierte el scan a puntos (x,y), extrae segmentos, identifica la pared derecha, analiza el frente y actualiza filtros EMA.

- `cb_odom(msg)`: extrae yaw (cuaternión→Euler) y velocidad lineal de la odometría, usados como apoyo opcional en giros y detección de bloqueo.
- `loop_control()`: temporizado a `control_rate_hz` (20 Hz por defecto). Aplica capa de emergencia, detección de 'stuck', ejecuta el estado activo de la FSM y publica velocidades con rampa de aceleración.

2.1 Corrección crítica documentada en el propio código (fix v2.1)

El encabezado del archivo documenta un bug de compatibilidad QoS: el driver del LiDAR MS200 publica `/scan` en `BEST_EFFORT`, mientras la suscripción usaba por defecto `RELIABLE`. Esa incompatibilidad no lanza ningún error en ROS 2: simplemente la suscripción nunca recibe mensajes y `have_scan` queda en `False` para siempre, dejando el robot inmóvil sin explicación visible. La solución aplicada usa `qos_profile_sensor_data` (`BEST_EFFORT`) y añade un 'watchdog' de arranque (`_scan_watchdog_check`) que emite un log de error explícito si no llega ningún scan en los primeros `scan_watchdog_s` segundos, indicando además los comandos de diagnóstico a ejecutar (`ros2 topic hz /scan`, `ros2 topic info /scan --verbose`).

3. Percepción LiDAR: cómo se detectan las formas (paredes, cajas y esquinas)

Esta es la sección central del informe. La detección de figuras NO usa visión por computadora: se basa enteramente en geometría sobre la nube de puntos del LiDAR 2D, mediante el algoritmo clásico Split-and-Merge, seguido de reglas geométricas de clasificación.

3.1 Paso 1 — Conversión de la nube de puntos al marco del robot

`_points_robot_frame()` recorre `msg.ranges` del `LaserScan` y descarta lecturas no finitas o fuera de `[range_min, range_max]`. Para cada punto válido calcula el ángulo real aplicando dos correcciones de calibración específicas del sensor MS200:

- `lidar_offset_deg` (180° por defecto): compensa que el 0° físico del sensor no coincide con el eje +x del robot.

- `lidar_y_sign` (-1.0 por defecto): corrige el sentido de giro/montaje del sensor respecto al eje Y del robot.

Con el ángulo corregido, convierte cada lectura polar (r, θ) a coordenadas cartesianas (x, y) en el marco del robot, donde $+x$ es 'adelante' y $+y$ es 'izquierda'. También permite excluir un arco angular fijo (`ignore_deg_min/max`), útil si el chasis del robot tapa parcialmente el sensor.

3.2 Paso 2 — Pre-segmentación por huecos

`_pre_seg()` corta la nube en grupos contiguos allí donde detecta un salto: un hueco de índice mayor a `sm_pre_gap_idx` (6 muestras) o un salto de distancia euclidiana mayor a `sm_pre_gap_dist` (0.40 m) entre puntos consecutivos. Esto separa de entrada objetos claramente distintos (p. ej. la pared y una caja separada de ella) antes de intentar ajustar rectas.

3.3 Paso 3 — Split-and-Merge iterativo (extracción de segmentos rectos)

Sobre cada grupo pre-segmentado se aplica el algoritmo Split-and-Merge, implementado de forma iterativa con una pila explícita (`_split_iterative`) para evitar desbordamientos de recursión con datos ruidosos:

- **SPLIT**: se traza la recta entre los dos extremos del grupo y se calcula, para cada punto intermedio, la distancia perpendicular a esa recta (`_perp_dist`). Si el punto más alejado supera un umbral, el grupo se corta en ese índice y el proceso se repite recursivamente sobre las dos mitades.
- **Umbral ADAPTATIVO al rango**: $\text{thresh} = \text{sm_split_thresh} * (1 + \text{sm_adaptive_range_k} * r_{\text{promedio}})$. Como el ruido angular del LiDAR se traduce en más ruido métrico a mayor distancia, el umbral de corte crece con el rango medio del segmento evaluado, evitando sobre-segmentar objetos lejanos.
- **MERGE multi-pasada** (`_merge_until_stable`): tras el split, se intenta fusionar segmentos adyacentes si, unidos, siguen formando una única recta válida (se reintenta el split sobre la unión y se comprueba que produce un solo segmento). Se repite hasta `sm_merge_max_passes` (5) o hasta que no cambie el número de segmentos, es decir, hasta convergencia.

Cada segmento final se valida con un mínimo de puntos (`sm_min_pts=3`) y longitud (`sm_min_len=0.06 m`), y se construye como un objeto `Segment` con:

Campo	Significado
p1, p2	Extremos (x,y) del segmento
length	Longitud euclidiana
mean_y	Y promedio de los puntos → distancia lateral
alpha	Ángulo del segmento respecto a +x (heading de la pared)
residual	RMS de la distancia perpendicular de los puntos a la recta ajustada → calidad del ajuste
n_pts	Número de puntos que forman el segmento

3.4 Paso 4 — Selección de la 'pared derecha' (`_best_right_segment`)

De todos los segmentos detectados en el frame, se elige el candidato a pared lateral derecha aplicando cuatro filtros geométricos en cascada:

- Longitud mínima (`sm_min_len`).
- Orientación: el segmento debe extenderse mayormente a lo largo de +x ($|\Delta x|/\text{longitud} > \text{wall_cos_lat_min} = 0.50$), descartando segmentos casi perpendiculares al avance (que serían más bien 'frente', no 'pared lateral').
- Lado derecho: `mean_y` por debajo de un umbral dinámico ligado a `target_wall_dist`, para no descartar paredes a las que el robot ya está muy próximo.
- Cercanía: entre los candidatos válidos, se toma el de `|mean_y|` mínimo (el más próximo lateralmente).

Importante: este mismo mecanismo es el que permite reutilizar la pared del anillo y la pared de una caja de forma indistinta. Durante la maniobra de rodeo (estado `BOX_BYPASS`), la cara de la caja pasa a jugar el rol de 'pared derecha' para el mismo controlador PID, sin lógica especial adicional.

3.5 Paso 5 — Análisis frontal: distancia + ANCHO ANGULAR (clave de la clasificación caja vs. esquina)

`_front_analysis()` recorre los puntos dentro de un sector frontal ($\pm \text{front_sector_deg} = \pm 38^\circ$) y agrupa lecturas consecutivas en 'arcos continuos' cuando cumplen dos condiciones simultáneas: continuidad radial (variación de distancia entre puntos consecutivos $< \text{front_arc_radial}$) y continuidad angular (separación angular entre

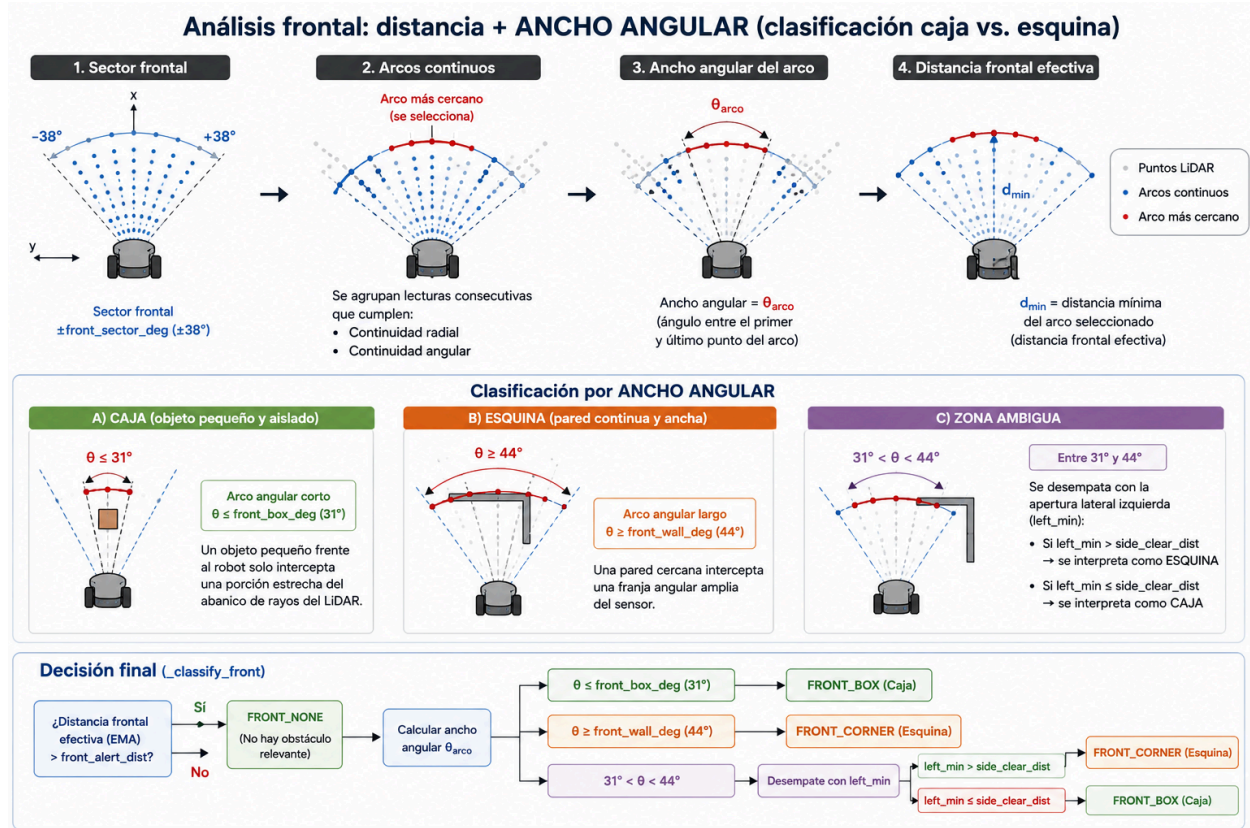
puntos consecutivos $< \text{front_arc_ang_gap}$). De todos los arcos continuos identificados, se retiene el que tiene la distancia mínima (el obstáculo más cercano), y se calcula su ANCHO ANGULAR total (ángulo entre su primer y último punto).

Esta es la métrica que distingue geométricamente una caja de una esquina de pared:

Tipo de obstáculo frontal	Ancho angular observado	Interpretación física
Caja (objeto pequeño y aislado)	Arco angular corto: $\leq \text{front_box_deg}$ (31°)	Un objeto pequeño frente al robot solo intercepta una porción estrecha del abanico de rayos del LiDAR
Esquina de pared (superficie continua y ancha)	Arco angular largo: $\geq \text{front_wall_deg}$ (44°)	Una pared cercana intercepta una franja angular amplia del sensor
Zona ambigua	Entre 31° y 44°	Se desempata con la apertura lateral izquierda (left_min): si hay espacio libre a la izquierda ($> \text{side_clear_dist}$) se interpreta como esquina; si no, como caja

Esta clasificación se materializa en `_classify_front()`, que primero comprueba si la distancia frontal efectiva (EMA) supera `front_alert_dist` (si es así, no hay obstáculo relevante: `FRONT_NONE`) y, de haberlo, decide entre `FRONT_CORNER` y `FRONT_BOX` según las reglas de la tabla anterior.

3.5.1. Análisis frontal: distancia + ancho angular



3.6 Suavizado temporal (EMA)

Tanto la distancia frontal como la distancia a la pared derecha se filtran con un promedio móvil exponencial (EMA) frame a frame ($\alpha_{\text{ema_front}}=0.40$, $\alpha_{\text{ema_wall}}=0.35$), para reducir el ruido de medición del LiDAR y evitar que jitter puntual dispare falsas transiciones de estado o inestabilidad en el control.

4. Control: wall-follow con PID completo + heading

El seguimiento de pared se resuelve con un controlador PID de dos términos proporcionales combinados (posición lateral + heading), más derivativo e integral opcionales:

$$w = -K_p \cdot (d_{\text{der}} - d_{\text{objetivo}}) - K_{\alpha} \cdot \alpha - K_d \cdot d(\text{PV})/dt(\text{EMA}) - K_i \cdot \int e \cdot dt$$

Término	Rol	Parámetro
Proporcional de distancia	Corrige el error lateral respecto a <code>target_wall_dist</code>	$K_{p_dist} = 1.5$

Proporcional de heading	Corrige la orientación relativa a la pared (alpha del segmento) — clave para evitar el zigzag típico de un PID de solo posición	$K_{\alpha} = 1.0$
Derivativo	Amortigua, calculado sobre la variable de proceso (no sobre el error) y filtrado con EMA, para evitar 'derivative kick' ante cambios bruscos de setpoint	$K_d_{\text{dist}} = 0.2$
Integral	Opcional (desactivado por defecto: $K_i_{\text{dist}}=0.0$), con zona muerta (K_i_{zone}) y clamp anti-windup ($K_i_{\text{max_accum}}$)	$K_i_{\text{dist}} = 0.0$

4.1 Velocidad lineal adaptativa (3 zonas)

`_adaptive_speed()` calcula v en función de la distancia frontal efectiva: velocidad de cruce completa si $fd \geq \text{speed_caution_dist}$ (0.50 m); interpolación lineal decreciente en la zona de cautela; y parada total ($v=0$) si $fd \leq \text{speed_critical_dist}$ (0.15 m).

4.2 Rampa de velocidad (anti-derrape)

`_publish_ramped()` limita la aceleración lineal y angular (max_v_accel , max_w_accel) antes de publicar en `/cmd_vel`, evitando saltos bruscos que provoquen deslizamiento de ruedas.

4.3 Giros en arco, no en pivote

Tanto en esquinas como en alineación frente a cajas se mantiene una velocidad lineal mínima ($v_{\text{turn_min}}$) durante el giro, en vez de pivotar sobre el eje, para conservar tracción en las ruedas.

5. Máquina de estados (FSM)

La navegación se organiza en 5 estados. Todas las transiciones automáticas (excepto emergencia) requieren `persist_frames` (3) lecturas consecutivas cumpliendo la condición, evitando cambios de estado por ruido puntual del sensor.

Estado	Función	Condición de salida
--------	---------	---------------------

CRUISE	Avanza siguiendo la pared derecha con el PID completo. Si pierde la pared más de wall_lost_frames, gira suavemente buscándola.	→ CORNER si frente clasifica como esquina; → BOX_ALIGN si clasifica como caja (ambos con persistencia)
CORNER	Arc-turn de $\sim 90^\circ$ en el sentido configurado (por odometría si hay yaw, si no por tiempo).	Ángulo alcanzado → CRUISE; corner_timeout_s → RECOVERY
BOX_ALIGN	Gira (arc-turn) hasta que el frente queda despejado durante align_clear_frames.	Frente limpio → BOX_BYPASS; align_timeout_s → RECOVERY
BOX_BYPASS	Wall-follow usando la cara de la caja como 'pared derecha'.	Pared larga estable ($\geq$ box_exit_wall_len durante box_exit_frames) → CRUISE; bypass_timeout_s → RECOVERY
RECOVERY	Retrocede (recovery_reverse_t) y luego gira (recovery_turn_deg), por odometría o por tiempo.	Maniobra completada o recovery_timeout_s → CRUISE

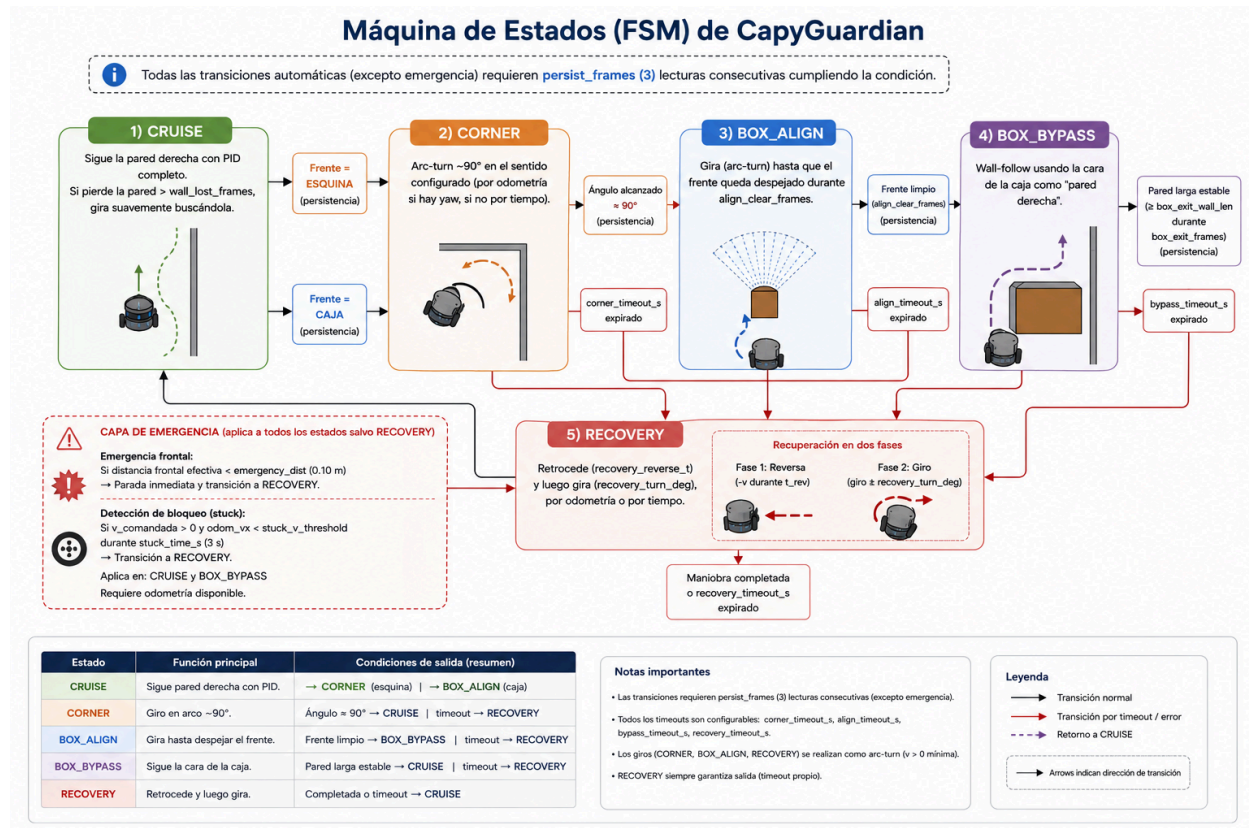
5.1 Capa de emergencia y detección de bloqueo (stuck)

- Emergencia: en cualquier estado salvo RECOVERY, si la distancia frontal efectiva cae por debajo de emergency_dist (0.10 m), se fuerza parada inmediata y transición a RECOVERY, sin esperar persistencia.
- Stuck: si el robot comanda $v > 0$ pero la velocidad real de odometría (odom_vx) permanece por debajo de un umbral durante stuck_time_s (3 s), se asume atasco físico y se entra en RECOVERY. Solo aplica en CRUISE y BOX_BYPASS, y requiere odometría disponible.

5.2 Recuperación en dos fases

_exec_recovery() ejecuta primero una fase de reversa (velocidad negativa durante recovery_reverse_t) y después una fase de giro (recovery_turn_deg), calculada por odometría si hay yaw disponible o, en su defecto,

5.2 Máquina de estados (FSM)



por temporización. Un timeout propio de RECOVERY garantiza que el robot nunca quede indefinidamente en este estado.

6. Parámetros configurables (resumen por categoría)

El nodo expone ~60 parámetros ROS 2 declarados en `__init__`, lo que permite recalibrar todo el comportamiento sin recompilar. Los más relevantes por categoría:

Categoría	Parámetros clave	Valores por defecto
Geometría LiDAR	lidar_offset_deg, lidar_y_sign, range_min/max	180°, -1.0, 0.05–3.5 m
Split & Merge	sm_split_thresh, sm_adaptive_range_k, sm_min_pts/len, sm_merge_max_passes	0.07, 0.25, 3 / 0.06 m, 5
Pared lateral	wall_min_len, wall_cos_lat_min	0.50 m, 0.50

Detección frontal	front_sector_deg, front_alert_dist, front_wall_deg, front_box_deg, side_clear_dist	38°, 0.42 m, 44°, 31°, 0.58 m
PID wall-follow	target_wall_dist, Kp_dist, K_alpha, Kd_dist, Ki_dist, max_ang_vel	0.30 m, 1.5, 1.0, 0.2, 0.0, 0.7
Velocidades	cruise_speed, v_turn_min, speed_caution/critical_dist	0.14 m/s, 0.04 m/s, 0.50/0.15 m
FSM / timeouts	persist_frames, corner_timeout_s, align_timeout_s, bypass_timeout_s, recovery_timeout_s	3, 8 s, 6 s, 15 s, 5 s
Emergencia / stuck	emergency_dist, stuck_v_threshold, stuck_time_s	0.10 m, 0.02 m/s, 3 s
Loop / diagnóstico	control_rate_hz, log_every_n, scan_watchdog_s, scan_qos_best_effort	20 Hz, 20, 5 s, True

7. Observaciones, fortalezas y riesgos

7.1 Fortalezas del diseño

- Detección de formas puramente geométrica (Split-and-Merge + ancho angular), ligera en cómputo, sin dependencias de visión ni modelos entrenados: adecuada para un Raspberry Pi 5.
- Reutilización elegante del mismo pipeline de percepción/control tanto para la pared del anillo como para el rodeo de cajas (BOX_BYPASS trata la caja como pared temporal).
- PID con heading, derivativo sobre PV y anti-windup: buenas prácticas de control que evitan zigzag y derivative kick.
- Máquina de estados robusta: persistencia de transición, timeouts por estado y capa de recuperación ante bloqueo o emergencia, con diagnóstico de arranque explícito (watchdog QoS).

7.2 Riesgos y puntos a vigilar

- La clasificación caja/esquina depende de tres parámetros fijos (front_box_deg, front_wall_deg, side_clear_dist) calibrados para geometrías conocidas del reto; cajas de tamaño distinto o pasillos más estrechos pueden requerir recalibración.

- El giro por 'timing' (cuando no hay odometría) en CORNER y RECOVERY es sensible a la fricción del suelo y a la carga de la batería; puede acumular error de orientación en recorridos largos.
- `dt` en `loop_control()` se toma como `dt_nominal` fijo ($1/\text{control_rate_hz}$) en vez de medir el tiempo real transcurrido entre ticks del timer, lo que puede introducir un pequeño desfase si el timer sufre jitter bajo carga de CPU.
- La detección de 'stuck' depende enteramente de `/odom`; en su ausencia, un atasco físico solo se resolvería vía la capa de emergencia frontal (si aplica) o quedaría sin detectar.
- El umbral adaptativo de Split-and-Merge y el ancho angular frontal no se recalculan dinámicamente según la velocidad del robot, por lo que a mayor velocidad de avance el margen de reacción efectivo se reduce proporcionalmente.

8. Conclusión

CapyGuardian resuelve el reto con una arquitectura clásica de robótica móvil, bien acotada y ampliamente parametrizada: percepción por Split-and-Merge para extraer geometría de línea, clasificación caja/esquina por ancho angular del arco frontal más cercano, control PID de posición+heading para el seguimiento de pared, y una FSM con persistencia, timeouts y recuperación para orquestar el comportamiento. El punto de mayor valor técnico es la reutilización del mismo concepto de 'pared' tanto para el anillo como para el rodeo de cajas, lo que simplifica notablemente el código evitando duplicar lógica de control específica por tipo de obstáculo.